


```
49         oi.Quantity,
50         oi.UnitPrice,
51         SubTotal = oi.Quantity * oi.UnitPrice // Tính tạm để tiện xem
52     })
53 }
54 .ToListAsync();
55
56 return Ok(orders);
57 }
58
59 // =====
60 // POST /api/orders
61 // Tạo đơn hàng mới
62 // Request body: { "items": [{"fruitId": 1, "quantity": 5}, ...] }
63 // =====
64 [HttpPost]
65 public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDTO dto)
66 {
67     // 1. Lấy userId từ JWT token
68     var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
69     if (userIdClaim == null)
70         return Unauthorized("Không tìm thấy thông tin người dùng trong token.");
71
72     int userId = int.Parse(userIdClaim);
73
74     // 2. Validate từng item trong đơn hàng
75     var orderItems = new List<OrderItem>();
76     decimal totalAmount = 0;
77
78     foreach (var item in dto.Items)
79     {
80         // Validate: quantity > 0
81         if (item.Quantity <= 0)
82             return BadRequest($"Quantity của FruitId {item.FruitId} phải lớn hơn 0.");
83
84         // Validate: fruitId tồn tại
85         var fruit = await _context.Fruits.FindAsync(item.FruitId);
86         if (fruit == null)
87             return NotFound($"Không tìm thấy Fruit với Id = {item.FruitId}.");
88
89         // Validate: stock đủ không
90         if (fruit.StockQuantity == null || fruit.StockQuantity < item.Quantity)
```

```
91         return BadRequest($"Fruit '{fruit.FruitsName}' không  
           đủ hàng trong kho. Còn lại: {fruit.StockQuantity ??  
           0}.");  
  
92  
93         // Lấy giá hiện tại của Fruit làm UnitPrice  
94         decimal unitPrice = fruit.Price;  
95         decimal subTotal = unitPrice * item.Quantity;  
96         totalAmount += subTotal;  
97  
98         orderItems.Add(new OrderItem  
99         {  
100             FruitId = item.FruitId,  
101             Quantity = item.Quantity,  
102             UnitPrice = unitPrice,  
103             CreatedAt = DateTime.Now  
104         });  
105  
106         // Trừ stock sau khi đặt hàng  
107         fruit.StockQuantity -= item.Quantity;  
108         _context.Fruits.Update(fruit);  
109     }  
110  
111     // 3. Tạo Order  
112     var order = new Order  
113     {  
114         UserId = userId,           // Lấy từ JWT  
115         OrderDate = DateTime.Now,  // Server tự set  
116         Status = "Pending",        // Mặc định  
117         TotalAmount = totalAmount, // Tính từ items  
118         CreatedAt = DateTime.Now,  
119         OrderItems = orderItems  
120     };  
121  
122     _context.Orders.Add(order);  
123     await _context.SaveChangesAsync();  
124  
125     return Ok(new  
126     {  
127         Message = "Tạo đơn hàng thành công.",  
128         OrderId = order.Id,  
129         order.UserId,  
130         order.OrderDate,  
131         order.Status,  
132         order.TotalAmount,  
133         OrderItems = order.OrderItems.Select(oi => new  
134         {  
135             oi.FruitId,  
136             oi.Quantity,  
137             oi.UnitPrice,
```

```
138         SubTotal = oi.Quantity * oi.UnitPrice
139     })
140 });
141 }
142
143 // =====
144 // GET /api/orders/user/{userId}
145 // Lấy đơn hàng theo userId
146 // Chỉ được xem đơn hàng của chính mình
147 // =====
148 [HttpGet("user/{userId}")]
149 public async Task<IActionResult> GetOrdersByUserId(int userId)
150 {
151     // Lấy userId từ JWT token
152     var userIdClaim = User.FindFirst                                     ➤
153         (ClaimTypes.NameIdentifier)?.Value;
154     if (userIdClaim == null)
155         return Unauthorized("Không tìm thấy thông tin người dùng ➤
156         trong token.");
157
158     int tokenUserId = int.Parse(userIdClaim);
159
160     // Validate: chỉ được xem đơn hàng của chính mình
161     if (tokenUserId != userId)
162         return Forbid(); // 403 - không có quyền xem của người ➤
163         khác
164
165     var orders = await _context.Orders
166         .Where(o => o.UserId == userId)
167         .Include(o => o.OrderItems)
168         .ThenInclude(oi => oi.Fruit)
169         .Select(o => new
170     {
171         o.Id,
172         o.UserId,
173         o.OrderDate,
174         o.TotalAmount,
175         o.Status,
176         o.CreatedAt,
177         OrderItems = o.OrderItems.Select(oi => new
178     {
179         oi.Id,
180         oi.FruitId,
181         FruitName = oi.Fruit.FruitsName,
182         oi.Quantity,
183         oi.UnitPrice,
184         SubTotal = oi.Quantity * oi.UnitPrice
185     })
186     })
187 }
```

```

184         .ToListAsync();
185
186         if (!orders.Any())
187             return NotFound($"Không tìm thấy đơn hàng nào của  
userId = {userId}.");
188
189         return Ok(orders);
190     }
191 }
192 }
193
194
195 ////////////////////////////////////////
196 ////////////////////////////////////////
197 using Microsoft.AspNetCore.Authorization;
198 using Microsoft.AspNetCore.Mvc;
199 using testFinal.DTOs;
200 using testFinal.Models;
201 using testFinal.Repositories;
202
203 namespace testFinal.Controllers
204 {
205     [ApiController]
206     [Route("api/[controller]")]
207     public class FruitsController : ControllerBase
208     {
209         private readonly IFruitRepository _repo;
210
211         public FruitsController(IFruitRepository repo)
212         {
213             _repo = repo;
214         }
215
216         // GET: api/fruits - Public
217         [HttpGet]
218         public async Task GetAll()
219         {
220             var fruits = await _repo.GetAll();
221             var result = fruits.Select(f => new FruitDTO
222             {
223                 Id = f.Id,
224                 Name = f.FruitsName,
225                 Price = f.Price,
226                 StockQuantity = f.StockQuantity,
227                 CategoryId = f.CategoryId,
228                 CategoryName = f.Category?.CategoriesName
229             });

```

```
230         return Ok(result);
231     }
232
233     // GET: api/fruits/{id} - Public
234     [HttpGet("{id}")]
235     public async Task<IActionResult> GetById(int id)
236     {
237         var fruit = await _repo.GetById(id);
238         if (fruit == null) return NotFound(new { message = "Fruit not found" });
239
240         return Ok(new FruitDTO
241         {
242             Id = fruit.Id,
243             Name = fruit.FruitsName,
244             Price = fruit.Price,
245             StockQuantity = fruit.StockQuantity,
246             CategoryId = fruit.CategoryId,
247             CategoryName = fruit.Category?.CategoriesName
248         });
249     }
250
251     // POST: api/fruits - Admin Only
252     [HttpPost]
253     [Authorize(Roles = "Admin")]
254     public async Task<IActionResult> Create(FruitDTO dto)
255     {
256         if (!ModelState.IsValid) return BadRequest(ModelState);
257
258         var fruit = new Fruit
259         {
260             FruitsName = dto.Name,
261             Price = dto.Price,
262             StockQuantity = dto.StockQuantity,
263             CategoryId = dto.CategoryId
264         };
265
266         await _repo.Add(fruit);
267         return Ok(new { message = "Created successfully" });
268     }
269
270     // PUT: api/fruits/{id} - Admin Only
271     [HttpPut("{id}")]
272     [Authorize(Roles = "Admin")]
273     public async Task<IActionResult> Update(int id, FruitDTO dto)
274     {
275         if (!ModelState.IsValid) return BadRequest(ModelState);
276
277         var fruit = await _repo.GetById(id);
```

```
278         if (fruit == null) return NotFound(new { message = "Fruit not  
found" });  
279  
280         fruit.FruitsName = dto.Name;  
281         fruit.Price = dto.Price;  
282         fruit.StockQuantity = dto.StockQuantity;  
283         fruit.CategoryId = dto.CategoryId;  
284  
285         await _repo.Update(fruit);  
286         return Ok(new { message = "Updated successfully" });  
287     }  
288  
289     // DELETE: api/fruits/{id} - Admin Only  
290     [HttpDelete("{id}")]  
291     [Authorize(Roles = "Admin")]  
292     public async Task<IActionResult> Delete(int id)  
293     {  
294         var fruit = await _repo.GetById(id);  
295         if (fruit == null) return NotFound(new { message = "Fruit not  
found" });  
296  
297         await _repo.Delete(id);  
298         return Ok(new { message = "Deleted successfully" });  
299     }  
300 }  
301 }  
302  
303 //////////////////////////////////////   
304 //////////////////////////////////////   
305 using Microsoft.AspNetCore.Mvc;  
306 using Microsoft.EntityFrameworkCore;  
307 using testFinal.DTOs;  
308 using testFinal.Models;  
309  
310 namespace testFinal.Controllers  
311 {  
312     [ApiController]  
313     [Route("api/[controller]")]  
314     public class CategoriesController : ControllerBase  
315     {  
316         private readonly ApplicationDbContext _context;  
317  
318         // Sử dụng trực tiếp DbContext hoặc qua Repository đều được  
319         public CategoriesController(ApplicationDbContext context)  
320         {  
321             _context = context;  
322         }
```



```
367 // 2. Tạo Claims (Thông tin người dùng lưu trong Token)
368 var claims = new[]
369 {
370     new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
371     new Claim(ClaimTypes.Email, user.Email),
372     new Claim(ClaimTypes.Role, user.Role.ToString())
373 };
374
375 // 3. Tạo Key bảo mật (đọc từ appsettings.json)
376 var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config
    ["Jwt:Key"]!));
377 var creds = new SigningCredentials(key,
    SecurityAlgorithms.HmacSha256);
378
379 // 4. Tạo Token
380 var token = new JwtSecurityToken(
381     issuer: _config["Jwt:Issuer"],
382     audience: _config["Jwt:Audience"],
383     claims: claims,
384     expires: DateTime.Now.AddHours(1),
385     signingCredentials: creds
386 );
387
388 return Ok(new { token = new JwtSecurityTokenHandler().WriteToken
    (token) });
389 }
390 }
```